

Programspråk

Kapitel 12: Concurrency mm.

Henrik Björklund



UMEÅ UNIVERSITY

Kodbibliotek vs. språkkonstruktioner

- Semaforer
- Monitorer
- Meddleanden
- Transactional memory
- Indikera vad som kan göras samtidigt

En datastruktur med två **atomära** operationer.

- **wait**: om räknaren är större än noll, räkna ner och fortsätt. Om räknaren är noll, blockera
- **signal**: Om ingen väntar, räkna upp med 1. Om någon väntar, starta någon av de blockerade processerna.

Enkelt: Kan implementeras med ett minimum av kod

Nackdel: programmeraren ansvarar för allt

Oftast en språkkonstruktion

Bara en process/tråd kan exekvera åt gången

Kommandon för att signalera till andra och blockera sig själv

Finns olika varianter, men fungerar som “synchronized” i Java

Oftast en språkkonstruktion

Bara en process/tråd kan exekvera åt gången

Kommandon för att signalera till andra och blockera sig själv

Finns olika varianter, men fungerar som “synchronized” i Java

Hur bra fungerar det egentligen? Låser man för mycket?

Enkelt att förstå: Processer skickar brev till varandra via postlåda, brevbärare och brevlåda

Synkront eller asynkront – eller båda

Till exempel Ada har stöd för detta. Kan också implementeras via bibliotek.

Problem: hastighet och resurser.

Transactional memory

Ungefär som transaktioner i databaser: En sekvens instruktioner betraktas som ett atomärt block. Om inte alla instruktioner blir körda rullas de föregående tillbaka.

Finns i vissa implementationer av existerande språk.

Kan implementeras i mjuk- eller hårdvara.

Avbrott - vad är det?

- Riktigt oväntade: Minnesfel, diskfel, datorn brinner, etc.
- Öväntat, men kan hända: Numeriskt overflow, nolldivision, etc.
- Förväntat: filslut, etc.

Dessa kan alla resultera i att datorn genererar ett **avbrott**: den upptäcker att något är fel och signalerar detta.

Vad göra?

- Försöka återhämta sig
- Avsluta programmet

Egentligen skulle det räcka med kod som kollar flaggor, returvärden, etc.

I praktiken blir det rörig kod som är svår att förstå.

De flesta moderna språk använder ungefär samma modell (de reserverade orden kan skilja sig åt): **try-catch**

- Tydligare och enklare kod
- Lättare att tänka på vilka typer av fel som kan uppstå
- Kan välja att hantera felen på olika nivåer

- Hur aktiveras ett undantag när det genereras i programmet?
- Finns det parametrar? Vad ska de innehålla
- Finns det möjlighet att hantera avbrottet på olika nivåer?
- Hur fungerar i så fall denna mekanism?
- Var ska felet hanteras?
- Kan det finnas flera felhanterare? Hur väljs den rätta ut?

Händelser (Events)

Events skiljer sig från avbrott i att de inte indikerar att något gått fel, utan är ett sätt att programmera.

De liknar avbrott, men har också stora likheter med meddelanden som de används mellan trådar.

Undantag och events har samma ursprung:
Hårdvarusignaler och avbrottsvektorer.

Javas event-hantering är ganska typisk, även om skillnader mellan språk förstås förekommer.

LISP - Scheme, ML, Haskell, Erlang

Utmärkande: Inga sidoeffekter

```
(DEFINE (member a lst)
  (COND ((NULL? lst) '())
        ((EQ a (CAR lst) #T)
         (ELSE (member a (CDR lst)))))
```

```
(DEFINE (member a lst)
  (COND ((NULL? lst) '())
        ((EQ a (CAR lst) #T)
         (ELSE (member a (CDR lst)))))
```

```
fun member a [] = false
  | membera(b::l)=a=b orelse member a l
```

- I princip samma sak
- Helt funktionellt (inga sidoeffekter)
- Bättre hantering av I/O
- Icke-strikt semantik -> [Lazy evaluation](#)

- I princip samma sak
- Helt funktionellt (inga sidoeffekter)
- Bättre hantering av I/O
- Icke-strikt semantik -> [Lazy evaluation](#)

```
ints = [0..]  
squares = [n*n | n <- ints]  
member 16 squares
```

- Bevisförning
- Programtransformationer
- Funktioner är första klassens medborgare
- Enklare syntax och semantik
- Naturlighet

Stöd i “icke-funktionella” språk

En del imperativa språk har stöd för en del funktionell programmering (tex funktioner som parametrar, anonyma funktioner).

- Java
- Javascript
- Python
- Ruby
- C#

Grundidé: Modellera en problemdomän logiskt

- Ställ upp regler
- Ange fakta
- Fråga om ett visst mål följer av reglerna och fakta
- Tala **inte** om för datorn hur den ska göra
- Detta sköts av en bevismotor

Exempel: Sortera en lista

- C, Java, ML, etc
 - Steg-för-steg-beskrivning av hur man sorterar
- Logikspråk
 - Beskriv egenskaperna hos en sorterad lista

Exempel: Sortera en lista

- C, Java, ML, etc
 - Steg-för-steg-beskrivning av hur man sorterar
- Logikspråk
 - Beskriv egenskaperna hos en sorterad lista

$$\text{sort}(\text{oldList}, nL) \subset \text{permute}(\text{oldList}, nL) \cap \text{sorted}(nL)$$
$$\text{sorted}(\text{list}) \subset \forall j : 1 \leq j < n, \text{list}(j) \leq \text{list}(j+1)$$

```
// FACTS
studies(charlie, csc135).
studies(olivia, csc135).

teaches(kirke, csc135).
teaches(collins, csc131).

// RULES
professor(X, Y) :- teaches(X, C), studies(Y, C).

// QUERIES / GOALS
?- studies(charlie, What)
?- professor(kirke, Students)
```

- Effektivitet
- Hur ska man söka?
- True/fail
- Negationer

För att i viss mån avhjälpa dessa problem använder vissa språk `cut (!)` för att delvis styra sökningen.

- Database management
- Expert systems